

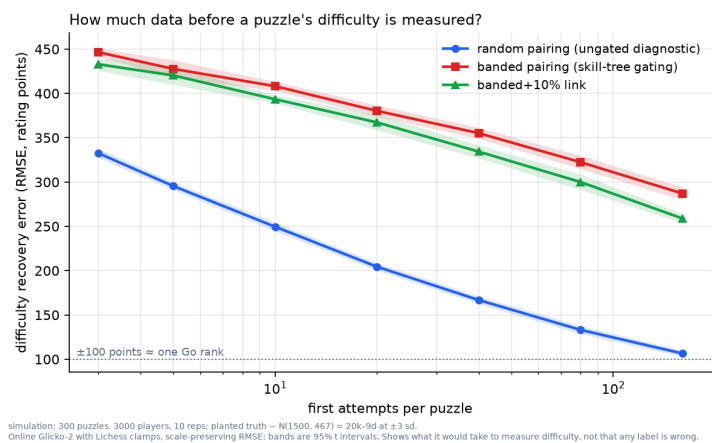
Measuring Go puzzle difficulty

How much attempt data would it take to measure a puzzle's difficulty from player results, instead of assigning it by hand?

What I did. Plant 300 puzzles and 3,000 players with known difficulties and skills. Simulate the first-attempt log that skill-tree gating produces — each player meets only puzzles near their own level, the exposure pattern implied by the tree on Go Magic's public page. Fit Glicko-2 on that log with the truth hidden, and score how far the fitted difficulties land from the planted ones. The scale: ± 100 rating points $\approx$ one Go rank; an error of 467 — the planted spread — would mean nothing was learned.

A puzzle's difficulty can be measured to about one rank from ~ 160 first attempts — but on skill-tree data only if the log is fitted *jointly*. Replaying the same log one attempt at a time, the standard online way, leaves almost three ranks of error. Same data: a 61% cut from changing nothing but how the fit is run.

The trap this avoids: running online Glicko-2 over your history, seeing ~ 290 points of error, and concluding the data is inadequate. The data is fine — the estimator is the problem. A joint refit of the identical log reaches ~ 110 .



Online Glicko-2 under three traffic shapes: random pairing (what a diagnostic test produces), skill-tree gating, and gating with 10% of traffic ungated. Lower is better; the dotted line is one-rank accuracy; bands are 95% intervals over ten repeats. The joint fit is in the table.

THREE RESULTS

The tree itself makes measurement harder

A skill tree serves each player puzzles near their own level, so no attempt ever directly compares an easy puzzle with a hard one. That costs **1.6 $\times$** the error at 10 attempts per puzzle, **2.7 $\times$** at 160 — and the gap stays near **2.8 $\times$** out to 1,280. Traffic does not close it.

The error is in the spacing, not the ordering

At useful volume the fit ranks puzzles almost perfectly — rank correlation **0.94** — while compressing the scale to under half its true width. It knows *which* puzzle is harder, not *by how much*. A standard, repairable measurement problem, not noise.

Uneven traffic is a second, separate cost

Everybody attempts the first node; few reach the last. At the same total volume that funnel adds **40–50** points of error and degrades the ordering — the one thing gating had left intact.

first attempts per puzzle	ungated	gated, online	gated, joint fit
10	250	408	388
40	167	355	252
160	107	287	112

Typical difficulty error in rating points ($\pm 100 \approx$ one rank). Means over ten simulated worlds; within each, the last two columns fit the identical log — only the estimator changes. 95% intervals are $\pm 3-8$ points; every contrast is significant.

WHAT TO DO ABOUT IT

Backfill jointly, serve online

Fit the existing history once, jointly, to get the labels. Keep online Glicko-2 for the live path, where updating one attempt at a time is the right tool.

Ship labels per puzzle, not per catalogue

Coverage will never be uniform, so show a measured label only where the attempts exist. Do not use Glicko's own confidence number (RD) as that switch: on gated data it claims **3.7 $\times$** more precision than it delivers.

Anchor the scale with an ungated test

A diagnostic that spans the whole rank range is the standard repair for a compressed scale: routing 25% of traffic through one buys back **57** points of error and much of the compression.

What this does not claim. It does not claim any hand-assigned label is wrong — checking that needs the private attempt log, which nobody outside the company has. It answers the question that comes first: if you measured difficulty from attempts, how much data would you need before the answer meant anything? That depends only on the estimator and the *shape* of the data, so simulation settles it. Nothing here uses private data.

The obvious next step is small: run the joint fit over one month of the real attempt log, first attempts only, and check whether the recovery curve matches this one.